Justificación de las decisiones técnicas

Proyecto Banco de Preguntas Saber

Equipo "Sin Nombre"

Laboratorio de Ingeniería de Software II

Universidad del Cauca

Popayán, octubre de 2026

1. Introducción

Este documento justifica las especificaciones técnicas y de gestión que el equipo definió para el Banco de Preguntas Saber. Para cada decisión se presentan las alternativas que se consideraron, las razones por las que se eligió la opción adoptada, sus desventajas y cómo el equipo piensa mitigarlas. También se explica por qué algunas herramientas y prácticas se descartaron.

El criterio general que guió las decisiones fue el contexto real del proyecto: un sistema académico de alcance acotado, desarrollado por un equipo pequeño, con tiempo limitado y con el objetivo adicional de aplicar patrones de diseño y arquitectura vistos en el curso (capas, Observer, pipeline de filtros).

2. Resumen de decisiones

Aspecto	**Decisión**	**Alternativas no elegidas**
Estilo arquitectónico	Monolito (front-back) en capas	Microservicios
Capas	Presentation, Service, Access, Domain + Infra transversal	Arquitectura plana o por paquetes sin separación
Plataforma	Java SE (JSE)	Jakarta EE, Spring Boot
Construcción	Maven	Gradle, Ant, compilación manual
Gestión del proyecto	Jira	Trello, GitHub Projects, tablero físico
Control de versiones	Git + GitHub	GitLab, Bitbucket, SVN
Pruebas	JUnit (unitarias) + Mockito; pruebas de integración	TestNG, pruebas manuales
Patrón para la GUI	Observer / Observador	MVC completo, Observer sin infraestructura propia
Validación estructural (HU03)	Pipeline de filtros (simulado con objeto)	Validaciones if/else en un solo método
Descartado	Spring Boot, tablero de tareas independiente, Burn Down Chart	—

3. Contexto: épicas del proyecto

El alcance funcional se organizó en tres épicas, que condicionan las decisiones técnicas:

- **HE1 – Usuarios:** registro y gestión de usuarios del sistema.
- **HE2 – Gestión del banco:** administración de las preguntas (crear, consultar, modificar, filtrar).
- **HE3 – Validación estructural:** verificación de que cada pregunta cumple el formato requerido (HU03), implementada como pipeline.

Son pocas épicas, con reglas de negocio claras y una interfaz que debe reaccionar a cambios en los datos. Esto favorece una arquitectura simple, bien separada y fácil de probar.

4. Justificación detallada

4.1. Arquitectura: monolito (front-back) frente a microservicios

- **Decisión:** Un solo artefacto desplegable que contiene presentación y lógica (front-back).
- **Alternativas consideradas:** Microservicios (por ejemplo, un servicio de usuarios y otro de banco de preguntas).

Por qué la elegimos:

- El sistema tiene solo tres épicas y un equipo pequeño; no hay módulos que requieran escalar de forma independiente.
- Un monolito se compila, ejecuta y depura como un único proyecto, lo que reduce el tiempo de configuración.
- Evita los costos de comunicación entre servicios (red, serialización, manejo de fallos parciales, versionado de contratos).
- Las pruebas de integración son más sencillas porque todo corre en el mismo proceso.

Desventajas y riesgos:

- Si el sistema crece mucho, todo el código se despliega junto y el acoplamiento puede aumentar.
- No se puede escalar un módulo por separado.
- Un error grave en un módulo puede afectar a toda la aplicación.
- **Cómo lo mitigamos:** se separa internamente por capas e interfaces, de modo que una futura extracción a servicios sea posible con poco esfuerzo.

4.2. Organización en capas: Presentation, Service, Access, Domain e Infra

- **Decisión:** Estructura `src` con los paquetes `presentation` (UserGUI), `access` (repositorios e interfaces), `domain` (Question, User e interfaces), `service` (Question-

Service, UserService e interfaces) e `infra` (Observer, Subject, Filter), además de `unit` y `app`.

- **Alternativas consideradas:** Una estructura plana, o una organización por funcionalidad sin capas definidas.

Por qué la elegimos:

- Separa responsabilidades: la interfaz no conoce los detalles de persistencia y el dominio no depende de la interfaz.
- El flujo de dependencias es claro (Presentation → Service → Access → Domain), lo que facilita entender y revisar el código.
- El uso de interfaces en cada capa permite sustituir implementaciones y simular dependencias con Mockito en las pruebas.
- Infra agrupa las piezas transversales (Observer, Subject, Filter) para que no contaminen las demás capas.
- Es una estructura estándar, fácil de explicar y de evaluar académicamente.

Desventajas y riesgos:

- Mayor cantidad de clases e interfaces para una funcionalidad sencilla (más "ceremonia").
- Riesgo de capas que solo delegan sin aportar lógica.
- Exige disciplina para no saltarse capas (por ejemplo, que la GUI acceda directamente al repositorio).
- **Cómo lo mitigamos:** revisiones de código en GitHub que verifiquen el sentido de las dependencias y pruebas unitarias por capa.

4.3. Plataforma: Java SE (JSE) frente a Jakarta EE y Spring Boot

- **Decisión:** Java SE como plataforma de desarrollo y ejecución.
- **Alternativas consideradas:** Jakarta EE (servidor de aplicaciones) y Spring Boot.

Por qué la elegimos:

- Los requisitos del proyecto no necesitan un servidor de aplicaciones ni servicios empresariales como transacciones distribuidas o mensajería.
- JSE es la base que domina todo el equipo y la que se usa en el curso, con lo que se reduce la curva de aprendizaje.
- Permite implementar manualmente los patrones (Observer, pipeline) y así demostrar su comprensión, en lugar de delegarlos a un framework.
- Menos dependencias: configuración mínima y arranque rápido.

Desventajas y riesgos:

- Hay que escribir a mano tareas que un framework resuelve (inyección de dependencias, persistencia, API REST, configuración).
- Menor cantidad de librerías listas para usar y comunidad de ejemplos menos orientada a este caso.
- Si en el futuro se requiere una API web o despliegue en la nube, habría que migrar o añadir un framework.
- **Cómo lo mitigamos:** el diseño por interfaces e inyección manual de dependencias deja la lógica independiente de la plataforma y facilita migrar después.

4.4. Construcción: Maven

- **Decisión:** Maven para compilar, gestionar dependencias y ejecutar pruebas.
- **Alternativas consideradas:** Gradle, Ant o compilar directamente desde el IDE.

Por qué la elegimos:

- Estructura estándar de proyecto y ciclo de vida conocido (compile, test, package), lo que unifica el trabajo entre integrantes.
- Gestiona de forma declarativa las dependencias (JUnit, Mockito) mediante el `pom.xml`, sin copiar archivos `.jar` a mano.
- Se integra bien con los IDE y con el resto del ecosistema Java; el equipo ya lo ha usado.
- Reproducibilidad: cualquier integrante obtiene el mismo resultado al clonar el repositorio.

Desventajas y riesgos:

- El `pom.xml` es extenso (XML verboso) y menos flexible que Gradle para casos personalizados.
- Los conflictos de versiones de dependencias pueden ser difíciles de diagnosticar.
- Requiere descargar dependencias al inicio y, a veces, conexión a internet.
- **Cómo lo mitigamos:** fijar versiones explícitas en el `pom.xml` y no versionar archivos generados.

4.5. Gestión del proyecto: Jira

- **Decisión:** Jira para registrar historias de usuario, épicas, tareas y su estado.
- **Alternativas consideradas:** Trello, GitHub Projects o seguimiento informal por chat.

Por qué la elegimos:

- Soporta de forma nativa la jerarquía épica → historia de usuario → tarea, que corresponde con HE1, HE2 y HE3 y sus historias (por ejemplo, HU03).
- Permite priorizar, asignar responsables y dejar trazabilidad de cada historia.

- Es una herramienta usada en la industria, por lo que el trabajo con ella tiene valor formativo.
- Se puede enlazar con GitHub para relacionar commits con historias.

Desventajas y riesgos:

- Su configuración es más compleja y tiene más funciones de las que un proyecto pequeño necesita.
- Los planes gratuitos tienen límites.
- Puede generar carga administrativa si se actualizan demasiado los tickets.
- **Cómo lo mitigamos:** usar un flujo mínimo de estados (por hacer, en curso, hecho) y actualizar los tickets al cerrar cada tarea.

4.6. Control de versiones: Git + GitHub

- **Decisión:** Git como sistema de versiones y GitHub como repositorio remoto.
- **Alternativas consideradas:** GitLab, Bitbucket o SVN.

Por qué la elegimos:

- Git es el estándar de facto; permite ramas, fusiones y trabajo en paralelo sin bloquear a otros integrantes.
- GitHub ofrece pull requests y revisión de código, útiles para mantener la calidad de las capas.
- Es gratuito para repositorios académicos y el equipo ya lo conoce.
- Sirve como respaldo y como evidencia del aporte de cada integrante.

Desventajas y riesgos:

- Los conflictos de fusión son frecuentes si varios editan los mismos archivos.
- Curva de aprendizaje para quien no domina los comandos de Git.
- Dependencia de un servicio externo y de la conexión a internet.
- **Cómo lo mitigamos:** ramas por historia de usuario, commits pequeños y pull requests revisados antes de integrar a la rama principal.

4.7. Pruebas: JUnit + Mockito y pruebas de integración

- **Decisión:** JUnit para pruebas unitarias, Mockito para simular dependencias y pruebas de integración entre capas.
- **Alternativas consideradas:** TestNG o pruebas manuales exclusivamente.

Por qué la elegimos:

- JUnit es el estándar de pruebas en Java y se integra directamente con Maven y los IDE.

- Mockito permite probar la capa Service sin depender de un repositorio real, aislando la lógica de negocio.
- La arquitectura por interfaces hace natural el uso de dobles de prueba.
- Las pruebas de integración verifican que las capas colaboran correctamente (por ejemplo, el pipeline de validación de punta a punta).
- Las pruebas automáticas detectan regresiones al integrar cambios de varios integrantes.

Desventajas y riesgos:

- Escribir pruebas consume tiempo y exige mantenerlas al cambiar el código.
- El abuso de mocks puede dar pruebas que pasan aunque la integración real falle.
- Las pruebas de integración todavía están sin definir, lo que es un riesgo de cobertura.
- **Cómo lo mitigamos:** priorizar pruebas sobre las reglas de negocio y el pipeline de HU03, y definir pronto un conjunto mínimo de pruebas de integración.

4.8. Patrón Observer para la interfaz gráfica frente a MVC

- **Decisión:** Observer/Observador (clases Subject y Observer en infra) para que la GUI se actualice cuando cambian los datos.
- **Alternativas consideradas:** MVC completo, o consultar los datos manualmente cada vez que se necesiten.

Por qué la elegimos:

- La interfaz necesita reaccionar a cambios (por ejemplo, al crear o modificar una pregunta) sin que el dominio conozca la GUI: el Subject notifica y los observadores se actualizan.
- Reduce el acoplamiento entre la lógica y la presentación, y respeta el sentido de dependencias entre capas.
- Permite añadir nuevas vistas u observadores sin modificar el sujeto.
- Es un patrón del curso y de implementación directa en JSE.

Desventajas y riesgos:

- Las notificaciones pueden volverse difíciles de rastrear si hay muchos observadores.
- Riesgo de actualizaciones en cascada o repetidas, y de fugas de memoria si no se desregistran los observadores.
- No organiza por sí solo toda la presentación como lo haría MVC (no define un controlador).
- **Cómo lo mitigamos:** limitar el uso de Observer a las notificaciones de la GUI y documentar quién se registra y cuándo. En el tablero MVC quedó anotado para revisión (“actualizar”); si el equipo lo adopta, se complementa con Observer en lugar de reemplazarlo.

4.9. Validación estructural (HU03): pipeline de filtros

- **Decisión:** Pipeline de filtros (Filter) que procesa cada pregunta en etapas; por ahora simulado con un objeto.
- **Alternativas consideradas:** Un único método con una cadena de condiciones.

Por qué la elegimos:

- Cada regla de validación es un filtro independiente, fácil de probar por separado.
- Se pueden agregar, quitar o reordenar reglas sin modificar las demás (principio abierto/cerrado).
- Encaja con la épica HE3 y con la capa Infra, donde se ubica la abstracción Filter.
- Simularlo con un objeto permite avanzar y probar sin depender aún de la capa de persistencia.

Desventajas y riesgos:

- Más clases y más infraestructura que un simple if/else.
- El orden de los filtros afecta el resultado y puede generar errores sutiles.
- Una cadena larga puede ser más difícil de depurar.
- **Cómo lo mitigamos:** definir un contrato claro para Filter, probar cada filtro por separado y probar el pipeline completo con casos válidos e inválidos.

5. Elementos descartados y por qué

El equipo decidió no incorporar los siguientes elementos. Esta decisión se tomó para concentrar el esfuerzo en lo que aporta valor al producto y a los objetivos académicos.

Elemento	Por qué no se usa	Consecuencia / desventaja
Spring Boot	Agrega una capa de magia (autoconfiguración, inyección automática) que oculta los patrones que se quieren practicar a mano; el alcance no justifica un framework completo.	Mayor trabajo manual y menos funcionalidades listas (API REST, persistencia, seguridad).
Tablero de tareas independiente	Jira ya concentra el seguimiento; mantener otro tablero duplicaría información.	Menos visibilidad rápida del estado si no se usa bien Jira.
Burn Down Chart	En un proyecto corto con pocas historias, el esfuerzo de estimar y actualizar puntos supera el beneficio.	No hay una medida visual del avance ni alerta temprana de retrasos.

Elemento	Por qué no se usa	Consecuencia / desventaja
Microservicios	Complejidad innecesaria para el alcance (ver 4.1).	Limita el escalado independiente en el futuro.

6. Resumen de desventajas y riesgos

Riesgo	Origen	Mitigación
Exceso de estructura para un sistema pequeño	Capas, interfaces y patrones	Mantener clases simples y evitar capas que solo deleguen
Trabajo manual repetitivo	No usar Spring Boot (JSE)	Reutilizar componentes de infra y automatizar con Maven
Cobertura de integración incompleta	Pruebas de integración sin definir	Definir pronto escenarios mínimos de integración
Conflictos de versiones del código	Trabajo paralelo en Git	Ramas por historia y pull requests
Falta de medición del avance	Sin Burn Down Chart	Revisar el estado de las historias en Jira cada semana
Notificaciones difíciles de rastrear	Patrón Observer	Documentar registros de observadores y limitar su uso

7. Aspectos pendientes por definir

- Alcance y escenarios de las pruebas de integración.
- Definición final del rol de MVC frente a Observer en la capa de presentación.
- Reemplazo del objeto simulado del pipeline de HU03 por la implementación real conectada a la capa Access.

8. Conclusión

Las decisiones priorizan simplicidad, separación de responsabilidades y facilidad de prueba por encima de la potencia de un framework o de una arquitectura distribuida. Se asume conscientemente el costo de escribir más código a mano y de no contar con ciertas herramientas de seguimiento, a cambio de un diseño claro, comprensible y alineado con los objetivos del curso.

Las desventajas identificadas son manejables con las mitigaciones propuestas y deben revisarse a medida que el proyecto evolucione.